Journal Title
XX(X):1–20

DOI: 10.1177/ToBeAssigned
www.sagepub.com/
SAGE

What a Football Event Feed Demands: Workload Characterisation and the End-to-End Staleness Budget for Real-Time Match Analytics

Gustavo Pedro Ricou[1]

Abstract

Real-time football analytics is built on an assumption nobody has measured: that the streaming infrastructure carrying event data is fast enough not to matter. We test it, and to do so we first characterise the workload itself. Using the StatsBomb open dataset across 2003–2023 (52 competition-seasons, 15 competitions, 3,315 matches, 2,806 men's and 509 women's), we describe what a live football event feed actually demands, then place measured broker latency inside the end-to-end staleness budget a decision-maker experiences.

Three findings follow. First, the workload is far burstier than its mean suggests: events arrive at 0.41 per second on average but reach 2.7 per second over a ten-second window, a peak-to-mean ratio of 6.5. Capacity planning from the mean — as the sports-analytics literature's stated latency budgets implicitly do — under-provisions by six-fold. Second, concurrency has a natural ceiling that can be measured rather than assumed. Kick-off schedules in the corpus produce at most 12 simultaneous matches and 21 in play at once, because domestic leagues deliberately synchronise final-matchday kick-offs; benchmarks sweeping hundreds of concurrent feeds are testing a regime football never reaches. Third, and decisively, transport is a rounding error in the budget. Against event annotation — performed by human operators and reported by vendors at seconds — our measured broker transport of 1.35 ms is 0.009% of end-to-end staleness, and the difference between Apache KAFKA and REDIS Streams is 0.0023%. For infrastructure choice to own even one percent of what a decision-maker experiences, annotation would have to complete in 134 ms, two orders of magnitude faster than a human pipeline can operate. The irrelevance of broker choice for football is therefore structural, not an artefact of our testbed.

The study is equally an account of how easily such a benchmark misleads. Six defects, each found after it had produced a publishable-looking result, are documented: a process-relative clock, a saturating replay speed, an asymmetric load generator, a concurrency axis that was covertly a throughput axis, a heavy-tailed load-generator artefact that contaminated every mean-based statistic, and cross-process timestamp inversion under accelerated replay that invalidated an entire corpus. We report the diagnostics that caught each, including a clock-integrity gate that condemns runs by stated rule, and a manipulation check adopted after one intervention silently failed to apply three separate times. The recurring lesson is that a co-located, accelerated testbed flatters the instrument rather than the system.

Keywords

Real-time sports analytics, football, in-play win probability, Age of Information, streaming systems, Apache Kafka, Redis Streams, latency benchmarking, decision quality, reproducible research

1 Introduction

The rapid digitization of sports has created an insatiable demand for real-time analytics. Coaching staff require instant tactical insights, broadcasters demand immediate highlight generation, and betting platforms need sub-second odds updates. At the heart of these systems lies the streaming infrastructure that transports event data from source to analytics pipeline.

Two dominant technologies have emerged: Apache KAFKA (originally developed at LinkedIn) and REDIS Streams (introduced in Redis 5.0). KAFKA is a distributed event streaming platform designed for high-throughput, fault-tolerant data pipelines. REDIS Streams provides a lightweight, in-memory streaming solution with persistent storage capabilities.

While numerous studies have compared these technologies, the literature lacks sports-specific evaluations. Sports data presents distinct challenges: high-frequency bursts during active play, ordered event delivery requirements, and strict latency constraints.

Contributions. This paper makes four contributions. (1) We pose and answer, on open data, the question of *when*

[1]School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland

Corresponding author:
Gustavo Pedro Ricou, School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland.
Email: pedrorig@tcd.ie

streaming-infrastructure latency actually degrades an in-play football decision, by coupling measured event-delivery latency to a calibrated in-play win-probability model through the Age-of-Information lens—to our knowledge the first infrastructure → decision-quality analysis in sports analytics. (2) We provide a fair, artifact-free Apache KAFKA vs. REDIS Streams latency benchmark on the StatsBomb open dataset, with fully open load-generation code. (3) We document six design defects—a process-relative clock, a saturating replay speed, an asymmetric load generator, a concurrency axis that was covertly a throughput axis, a heavy-tailed load generator that contaminated every mean, and cross-process timestamp inversion under accelerated replay—each of which produced a publishable-looking result before it was caught, offering a reusable protocol for honest streaming benchmarks. (4) We identify the condition that actually decides the question, and the mechanism behind it: with consumers co-located the two backends are statistically equivalent, but a *round-trip-bound consumption pattern*—acknowledging each message and awaiting the reply—caps a consumer's drain rate near $1/\mathrm{RTT}$, so ordinary network latency turns that equivalence into a factor-of-275 gap in decision staleness. We test the mechanism by intervention: batching the acknowledgements is a $40 times$ improvement, replicated with almost no variance, and read-loop instrumentation locates the ceiling in the acknowledgement path rather than the read path as we had first supposed. What the evidence supports throughout is that the consumption pattern and the deployment topology, not the choice of product, are the design variables.

1.1 Sports-Specific Latency Requirements

Real-time sports analytics imposes domain-specific latency constraints that vary by use case and stakeholder. As summarized in Table 1, different applications demand fundamentally different performance characteristics. Betting platforms require sub-500ms latency for in-play odds updates, as even a 2-second delay can erode user trust and cost millions in missed wagers Solutions (2025); Ververica (2025). Broadcasting applications can tolerate up to 3 seconds for live video synchronization OptiView (2025); Promwad (2025), while coaching decision systems need <500ms to enable real-time tactical adjustments Pappas et al. (2020); Sports (2023).

The concept of an "actionability window" formalizes this requirement: the maximum latency within which insights can still influence decisions. These budgets, summarised in Table 1, are however *asserted* by practitioners rather than derived: they say how fast a pipeline should be, not what missing the budget costs. That is precisely the gap RQ4 addresses. Our decision-staleness metric replaces the assumed threshold with a measured quantity—how much probability mass a decision-maker holds incorrectly, and for how long—so the question becomes not "did we meet 500 ms?" but "how wrong was the forecast, and for how long, because we did not?" We therefore use Table 1 as the practitioner context our results are interpreted against (Section 5), not as the study's success criterion.

1.2 Research Questions

This study addresses five research questions, culminating in the decision-quality question that motivates the paper and the deployment condition that governs it:

RQ1 (single feed): For a single live match feed, does the choice of streaming backend (Apache KAFKA vs. REDIS Streams) materially affect Time-to-Insight (TTI)?

RQ2 (concurrency): How does TTI scale as one broker host serves $N \in \{1, 5, 10\}$ concurrent match feeds, each carrying a distinct match, and do the two backends differ?

RQ3 (fairness and durability): With payload and protocol overheads controlled for fairness, what latency cost do stronger durability settings (KAFKA `acks=all`; REDIS `appendfsync always`) impose?

RQ4 (decision degradation): When measured delivery latency is translated into an in-play win-probability estimate through the Age-of-Information lens, does it degrade an in-play football decision, and in which regime does the effect become material?

RQ5 (deployment topology): Does any equivalence established with co-located components survive when the consumer sits a network hop from the broker—and if not, what mechanism explains the difference?

These questions are motivated by gaps in the literature. Industry benchmarks suggest REDIS offers 2–5× lower latency than KAFKA Diaries (2025); Yerrapragada (2025), but there is no sports-specific empirical validation, and—as our measurement-validity analysis shows—such benchmarks are easily confounded. The CAP theorem Brewer (2012) and its extension PACELC Abadi et al. (2012) predict consistency–latency trade-offs (RQ3) that have not been quantified for these systems on sports workloads. Most importantly, while in-play win-probability models and the Age-of-Information view of staleness both exist (Section 2), no prior work connects *measured* infrastructure latency to in-play sports *decision* error—the gap RQ4 fills.

1.3 Hypotheses

For each research question we state null and alternative hypotheses. Where an industry "prior expectation" exists we name it explicitly, because a central finding of this paper is that the corrected data *refutes* the widely held prior—and that the uncorrected data appeared to confirm it only through instrumentation artifacts (Section 4.6).

RQ1: Single-feed architecture impact

H_{01}: $\mu_{\mathrm{TTI,KAFKA}} = \mu_{\mathrm{TTI,REDIS}}$ (no difference at $N{=}1$).

H_{11}: the two differ.

Test: Mann–Whitney U (Holm–Bonferroni corrected). **Prior expectation:** industry benchmarks Diaries (2025); Yerrapragada (2025) suggest REDIS is 2–5× faster, i.e. a large difference favouring REDIS. **Result (Section 5.2):** the corrected single-feed data does *not* reject $\mathbf{H_{01}}$ ($p = 0.69$)—the backends are at parity, refuting the prior.

Table 1. Sports Analytics Latency Requirements by Use Case and Stakeholder

Use Case	Stakeholder	Latency Range	Critical Threshold
Betting Platforms			
Live Odds Update	Betting Platform	100–500ms	$<$ **500ms**
In-Play Betting	Betting Platform	50–200ms	$<$ **200ms**
Broadcasting			
Live Video Sync	Broadcaster	500–3000ms	$<$ **3000ms**
Live Stats Overlay	Broadcaster	100–1000ms	$<$ **1000ms**
Interactive Features	Broadcaster	200–500ms	$<$ **500ms**
Coaching			
Tactical Adjustment	Coach	200–800ms	$<$ **800ms**
Player Substitution	Coach	500–1500ms	$<$ **1500ms**
Fan Applications			
Push Notifications	Fan	1000–3000ms	$<$ **3000ms**
Live Updates	Fan	500–2000ms	$<$ **2000ms**
Post-Match Analysis			
Initial Analysis	Analyst	5000–10000ms	$<$ **10000ms**

RQ2: Concurrency scaling

H_{02}: TTI is independent of the number of concurrent feeds N.

H_{12}: TTI increases with N, and the rate of increase differs by backend.

Test: Kruskal–Wallis across N; per-N Mann–Whitney. **Prior expectation:** both systems are marketed as horizontally scalable, suggesting near-constant TTI. **Result (Section 5.3):** the answer depends on the metric, and we report both. On TTI, $\mathbf{H_{02}}$ is not rejected ($p = 0.70, 0.20$). On broker *transport*, which is not swamped by load-generator scheduling lag, $\mathbf{H_{12}}$ holds decisively: REDIS rises 34% across N ($p = 6.7 \times 10^{-12}$) while KAFKA is flat (+0.3%). The effect is real but sub-millisecond, so the backends remain *equivalent* within the pre-specified margin. Separately, an earlier design in which every feed replayed the same merged ten-match plan exaggerated this into a large effect, because its concurrency axis was covertly a throughput axis (Table 6).

RQ3: Fairness and durability cost

H_{31}: $\mu_{\text{TTI, acks=all}} > \mu_{\text{TTI, acks=1}}$ (KAFKA: stronger durability costs latency).

H_{32}: $\mu_{\text{TTI, appendfsync always}} > \mu_{\text{TTI, everysec}}$ (REDIS: same).

Test: Mann–Whitney U, after verifying that payload size and (de)serialization overhead are equivalent across backends so the comparison is fair. **Expected and found:** both hold.

RQ4: Decision degradation (the contribution)

H_{04}: decision-staleness (Section 4.4) is independent of backend and concurrency—infrastructure latency does not translate into in-play decision error.

H_{14}: decision-staleness grows with delivery latency and diverges by backend under concurrency.

Test: per-N Mann–Whitney and Kruskal–Wallis across N on the Age-of-Information decision-staleness, plus TOST against the margin below. **Result (Section 5.4):** with components co-located, $\mathbf{H_{04}}$ survives and is strengthened into a positive equivalence claim—staleness is ≈0.014 probability-seconds at every N, statistically *equivalent* between backends, with no concurrency effect. $\mathbf{H_{14}}$ holds only once the consumer sits a network hop away (RQ5), where the gap reaches a factor of 275. Decision degradation is thus governed by deployment topology, not by backend or concurrency.

Equivalence Testing Our central claim is a *negative* one, and a non-significant difference test does not support it: failing to reject equality is not evidence of equality, particularly at modest sample sizes. We therefore state the claim with two one-sided tests (TOST), which invert the burden of proof by testing $H_0 : |\mu_{\text{KAFKA}} - \mu_{\text{REDIS}}| \geq \delta$ against $H_1 : |\mu_{\text{KAFKA}} - \mu_{\text{REDIS}}| < \delta$; rejecting H_0 establishes equivalence within the margin δ. Equivalently, the 90% confidence interval of the difference must lie entirely inside $(-\delta, +\delta)$, and we report both. The margin is pre-specified on domain grounds rather than fitted to the data: one broadcast video frame at 25 fps (40 ms) for latency, and the staleness a maximal event (total-variation shift of 1.0) accrues over one frame (0.04 probability-seconds) for decision-staleness. A difference smaller than a single frame cannot change what a coach, broadcaster or model does.

The margin is generous relative to the measurements—both backends deliver in roughly ten milliseconds, so 40 ms is about four times the quantity being compared—and we regard that as a limitation rather than a strength. Equivalence within one frame is a claim about *operational* indistinguishability, not about the systems being identical; a reader operating under a tighter budget should re-test at their own margin, and Section 5.3 shows that on broker transport a strictly smaller margin would *not* be met. We report the margin, the observed difference and the interval in every case so the reader can apply their own threshold.

RQ5: Round-trip-bound consumption (mechanism) The two clients consume in structurally different ways, and we hypothesise that this—not broker throughput—governs their behaviour once a network hop is present. The REDIS consumer issues a blocking `XREADGROUP` and cannot issue the next read until the previous reply returns; because the call returns as soon as *any* entry is available, at realistic arrival rates it retrieves roughly one event per round trip irrespective of its `COUNT` ceiling. Its sustainable drain rate is therefore about $1/\mathrm{RTT}$ events per second. The KAFKA consumer instead serves `poll()` from a locally prefetched buffer filled by pipelined fetch requests against a long-polling broker, so its drain rate is decoupled from RTT.

H_{05}: delivery latency rises additively with network delay for both systems (a fixed per-event cost).

H_{15}: REDIS is queue-stable only while the per-feed arrival rate $\lambda \lesssim 1/\mathrm{RTT}$; above that threshold its consumer cannot drain as fast as events arrive and latency is governed by an accumulating backlog rather than by the delay itself, whereas KAFKA remains stable at all tested delays.

The proximate cause in our implementation is an acknowledgement issued per message: the consumer calls `XACK` for each entry and waits for the reply before continuing, so even a read that returns two hundred entries is followed by two hundred sequential round trips. This is the idiomatic pattern in client tutorials, and on loopback it is free — measurably so: a `PING` round trip on an established connection to our REDIS container has a median of 0.46 ms (Section 4), a ceiling of ≈2,170 exchanges per second against a demand of $\lambda \approx 53$. The pattern has forty-fold headroom until the round trip becomes real, at which point the same code has none.

This yields four falsifiable predictions. (P1) A critical round-trip time near $1/\lambda$: at our $\lambda \approx 53$ events/s per feed, $\mathrm{RTT}_{\mathrm{crit}} \approx 19$ ms. (P2) Below it, per-event latency is *flat* across a run; above it, latency *grows* as the run proceeds. (P3) KAFKA shows no such growth at any tested delay. (P4) Amortising the acknowledgements over many messages should remove the bound entirely and restore parity, since it is the only per-event round trip in the loop. All four are tested in Section 5.5; P4 is the decisive one, because it is an intervention rather than an observation—if batching the acknowledgements does not help, the account is wrong.

We state that criterion before reporting the outcome, and we hold ourselves to it. All four predictions are borne out. A first multi-host attempt appeared to refute P4, but that test ran under accelerated replay which fails the clock-integrity gate of defect 6 and saturates the driver; repeated at $N{=}1$ under true real-time replay it confirms P4 decisively — a $40\times$ improvement, replicated with almost no variance (Section 5.6). The criterion is therefore met. The lesson is narrower than the one we first drew from it: a null obtained under a saturated instrument is not evidence against a hypothesis, and we had briefly reported it as though it were.

Statistical Framework All hypotheses are tested using the framework in Section 4.8: Holm–Bonferroni correction for multiple comparisons (Family-Wise Error Rate at $\alpha = 0.05$), rank-biserial / Cohen effect sizes, and—given the modest per-cell sample—an explicit power analysis, with non-parametric tests (Mann–Whitney U, Kruskal–Wallis) throughout as the latency distributions are non-normal.

2 Literature Review

Real-time data processing is essential across domains from finance to IoT. Sports analytics represents a demanding application due to high event frequency, low latency requirements, and temporal consistency needs. We review four strands—streaming platforms, how they are benchmarked, real-time sports systems, and the two literatures this paper joins—and identify the gap each leaves.

Streaming platforms. Stonebraker et al. Stonebraker et al. (2005) set out eight requirements for real-time stream processing, among them keeping data moving and processing with predictably low latency; these remain the yardstick against which streaming middleware is judged. KAFKA Kreps et al. (2011) realises them as a partitioned, replicated distributed commit log: throughput is won through batching and sequential disk writes, and parallelism through partitions consumed by independent consumers. REDIS Streams Sanfilipe (2017) takes the opposite design position, an in-memory append-only structure with consumer groups, trading capacity for minimal per-operation delay. The architectural consequence matters for this study: KAFKA's partitioning admits genuine intra-broker parallelism, whereas a REDIS node executes commands on a single thread, so concurrent streams contend for one execution context. The CAP theorem Brewer (2012) and PACELC Abadi et al. (2012) frame the resulting trade-offs—PACELC's "else" clause, that even absent partitions a system trades latency against consistency, is what our durability comparison (RQ3) probes.

How streaming systems are benchmarked—and how that goes wrong. Performance evaluations of KAFKA are numerous He et al. (2020); Gai and Qiu (2020), as are cross-framework comparisons of stream processors Carbone et al. (2015). Several works compare KAFKA directly against REDIS Streams Pandey et al. (2021); Zhang and Lee (2022), and grey-literature benchmarks routinely report REDIS as several times faster Diaries (2025); Yerrapragada (2025); JusDB (2025). A recent survey of benchmark practice in KAFKA event-streaming systems Anonymous (2025) observes that reported results are frequently not comparable, because client configuration, acknowledgement semantics and measurement instrumentation differ between the systems under test. Our own experience is a strong instance of that critique: six separate design defects (Section 4.6) each reversed the apparent winner, and the most subtle of them was precisely an asymmetric client configuration. Zhang et al. Zhang et al. (2021) propose time-to-insight as an evaluation metric for streaming analytics; we adopt TTI but decompose it into producer scheduling lag and transport, which is what makes the defects visible at all. What this literature does not supply is any link from the milliseconds it measures to a consequence in an application domain.

Established streaming benchmarks, and why none of them answers this question. Stream-processing systems have a mature benchmark literature, and it is worth being explicit about what it does and where this study sits relative

to it. Linear Road Arasu et al. (2004) is the canonical application benchmark, posing toll and traffic queries over a simulated vehicle stream; NEXMark Tucker et al. (2008) models an auction site and has become the standard query suite for streaming dataflow engines; ESPBench Hesse et al. (2021) constructs an enterprise workload spanning transactional and analytical operators; RIoTBench Shukla et al. (2017) supplies real-time IoT dataflows with realistic sensor arrival patterns; and the OpenMessaging Benchmark OpenMessaging Project, Linux Foundation (2018) is the closest analogue to what we do, exercising broker throughput and latency directly across messaging systems.

Three things follow for the present work. First, every one of these drives its system with a *synthetic* arrival process — generated vehicles, auctions, sensor readings or fixed-rate publishers — chosen for analytical tractability rather than fidelity to any domain. That is appropriate for a general-purpose benchmark and wrong for the question asked here, which is whether a *specific* domain's traffic stresses a broker; Section 3 therefore derives the arrival process, the burst structure and the concurrency from the sport itself. Second, none of them attaches a domain consequence to the milliseconds it reports: Linear Road scores by supported expressways, OpenMessaging by achieved throughput at a latency target. The staleness budget of Section 3.3 is our attempt to supply the missing step for one domain. Third, and most usefully for a reader wanting to reproduce or extend this, the OpenMessaging framework would be the right vehicle for the broker comparison alone — our contribution is not a better broker benchmark but the workload and the budget that tell you whether to run one at all.

We also note what we do *not* inherit from that literature: its scale. These benchmarks are built to find the breaking point of a distributed system. Football, as Section 3.2 shows, produces at most a couple of dozen concurrent feeds at a few events per second each. The interesting question for this domain is not where the system breaks but whether it is anywhere near breaking, and the answer is that it is not.

Real-time sports data systems. Sports analytics has moved from retrospective summary to live decision support Wright et al. (2022), driven by richer capture—player tracking and automated event detection Link et al. (2021)—and by openly available event data such as the StatsBomb corpus StatsBomb (2023) used here. Architectural accounts of live football pipelines Pappas et al. (2020); Sports (2023) specify latency budgets for coaching and broadcast, and the betting industry is explicit that in-play latency is commercially decisive Solutions (2025); Ververica (2025). These sources establish that latency *matters*, but they assert budgets rather than measuring what a given budget costs in decision quality; the thresholds are stated, not derived.

In-play win probability. Estimating a team's probability of winning *during* a match is an established sports-analytics task. For soccer, the state of the art is the Bayesian in-play model of Robberechts et al. Robberechts et al. (2021), which conditions on game state (score differential, time remaining, red cards, and rolling measures of chance creation) and is calibrated on large event corpora. Such models, however, are uniformly built and evaluated under the assumption that event data arrives *instantly*; none accounts for the latency of the pipeline that delivers events to the model. Because the state-of-the-art model's implementation is not public, we use a transparent, calibrated proxy on StatsBomb-derivable game state (Section 4.4).

Age of Information. The Age-of-Information (AoI) literature formalizes the cost of *stale* data: a monitor acting on information of age L incurs an error that grows with L Kaul et al. (2012). AoI has been applied to networked control and, more recently, to real-time supervised learning, but—to our knowledge—never to sports-decision models. Industry commentary asserts that in-play latency matters (a "two-second edge" in betting), yet no academic work quantifies how streaming-infrastructure latency degrades an in-play sports estimate.

The gap. Each strand stops short of the others. Streaming benchmarks measure latency but attach no domain meaning to it; sports systems assume latency budgets without deriving them; win-probability models assume instantaneous data; AoI supplies a cost-of-staleness formalism that has never been pointed at sport. This paper joins them: it treats the win-probability shift caused by a decisive event as the quantity that goes stale, and integrates it against *measured* delivery latency to obtain a decision-staleness cost—turning an infrastructure number into a sports one.

3 What a Football Event Feed Demands

Latency budgets for sports analytics are asserted rather than derived (Table 1), and streaming benchmarks are conducted at loads chosen for convenience. Before asking whether infrastructure can keep up, the workload has to be described. This section does that across the modern StatsBomb corpus: 52 competition-seasons spanning 2003–2023, 15 competitions, 3,315 matches, of which 2,806 are men's and 509 women's. To our knowledge no published characterisation of football event-feed arrival statistics exists, and it is a prerequisite for every claim that follows.

3.1 Arrival rate and burstiness

Football event data is sparse: a match yields a median of 3,507 events over roughly 8,412 seconds of match clock, a mean arrival rate of **0.415 events per second**. This is the figure the streaming literature would use to size a pipeline, and it is misleading. Measured over a sliding ten-second window, the same feeds peak at **2.7 events per second** — a peak-to-mean ratio of **6.45** (Table 2). Half of all inter-arrival gaps are at or below one second, and the fifth percentile is indistinguishable from zero: events cluster, because football clusters. A restart, a sequence of quick passes, a goal and its surrounding play all emit in tight groups separated by long quiet periods.

The practical consequence is that a buffer sized from the mean is undersized by more than a factor of six. This does not make football a demanding workload in absolute terms — 2.7 events per second is trivial for any modern broker — but it does mean the sparse-feed argument must be made on the peak, not the average, and no prior statement of football latency requirements does so.

Table 2. Football event-feed arrival statistics, medians across matches. The peak-to-mean ratio is the quantity that sizes a buffer, and it is six times the figure a mean-rate calculation would give.

Scope	Matches	Mean rate (ev/s)	Peak rate (ev/s)	Peak/mean
All competitions	3,315	0.415	2.70	6.45
La Liga	867	0.434	2.70	6.31
Ligue 1	435	0.430	2.70	6.32
Premier League	418	0.407	2.60	6.50
Serie A	380	0.423	2.70	6.35
1. Bundesliga	340	0.423	2.70	6.42
FA Women's Super League	326	0.397	2.60	6.55
FIFA World Cup	128	0.404	2.65	6.61
Women's World Cup	116	0.386	2.60	6.82
Indian Super League	115	0.346	2.60	7.64
African Cup of Nations	52	0.342	2.55	7.89

Peak rate is the maximum over any sliding ten-second window. Computed over all 3,315 matches. The sparsest competitions are the burstiest – the African Cup of Nations and Indian Super League have the lowest mean rates and the highest peak-to-mean ratios – so a low average is not a licence to provision for the average.

3.2 Concurrency has a ceiling, and it can be measured

Every concurrency level this literature benchmarks is chosen by hand. Our own earlier design swept $N \in \{1, 5, 10, 20\}$ and later $N \in \{10, 25, 50, 100\}$, which makes "does latency scale with concurrency?" a question about an invented variable. Football supplies the variable directly: match records carry a kick-off date and time, so the number of matches in play simultaneously is an empirical quantity.

Across the corpus there are 2,346 distinct kick-off instants. The largest carries **12 simultaneous matches**, and allowing for a nominal 115-minute duration the peak number of matches *in play* at any instant is **21**. The maxima are not accidents of sampling: domestic leagues schedule the final matchday simultaneously to protect competitive integrity, so a 20-team league plays ten concurrent matches by rule, and our largest observed slots are exactly such fixtures (for example 8 May 2016, La Liga and the Premier League).

Two consequences follow. Benchmarks that sweep hundreds of concurrent feeds — including our own earlier $N{=}100$ condition — are characterising a regime football does not produce. And the concurrency levels worth testing are derivable rather than arbitrary: the median, upper-quartile and maximum observed occupancy, together with the structural league bound, give $N \in \{1, 9, 10, 12\}$, which is what Section 5 replays.

One caveat is important and we state it wherever the number appears. StatsBomb open data is a *sample* of matches, not a complete record of any league-season, so observed simultaneity is a **lower bound**. The structural bound — half the teams in a league — is the safer planning figure, and for the largest European leagues it is ten.

3.3 The end-to-end staleness budget

A benchmark measures transport. A decision-maker experiences the sum of everything between the action on the pitch and the model's ability to act on it:

$$\text{staleness} = \underbrace{\text{annotation}}_{\text{human coding}} + \underbrace{\text{transport}}_{\text{measured here}} + \underbrace{\text{inference}}_{\text{model}}.$$

Live football event data is hand-coded by trained operators watching the match, a process whose inter-operator reliability has been studied Liu et al. (2013) but whose *latency* has not been measured in the peer-reviewed literature. Vendor descriptions of these pipelines report figures in the seconds. Because those are claims rather than measurements, we decline to adopt a point estimate and instead sweep annotation latency across 1–30 s, reporting what share of the budget each term owns throughout.

At the typical vendor-reported 15 s, our measured broker transport of 1.35 ms is $\mathbf{0.009}\%$ of end-to-end staleness, and the difference between the two backends is 0.35 ms, or $\mathbf{0.0023}\%$. Inverting the relation is more informative than either number: for transport to own even *one percent* of what a decision-maker experiences, annotation would have to complete in 134 ms; for ten percent, 12 ms. Both are two orders of magnitude beyond what a human coding pipeline can achieve.

This is the paper's central quantitative claim, and it is deliberately structural. We are not reporting that two brokers happened to perform similarly on our hardware; we are showing that the football workload sits four orders of magnitude below the point at which the choice becomes visible, and stating exactly what would have to change for that to stop being true. A faster annotation pipeline — automated event detection from tracking data, for instance — is the development that would make this paper's question live again.

4 Methodology

4.1 System Architecture

Our benchmark system comprises four components:

- Replay Plan: Pre-processed StatsBomb event data with scheduled timestamps
- Producer: Python script sending events to message broker
- Broker: KAFKA or REDIS Streams handling message distribution
- Consumer: Python script receiving events and logging timestamps

4.2 Dataset and Replay

We use the StatsBomb open dataset (1. Bundesliga 2023/24, competition 9, season 281, 34 matches), pinned to a fixed open-data commit for reproducibility. Each match is preprocessed into a CSV *replay plan* recording, per event, its scheduled emission offset. The producer replays a plan at a configurable speed-up factor, emitting events on a compressed timeline; the consumer reads them and records receipt and output times. For the headline latency results we replay at a non-saturating $10\times$ (Section 4.6) so that neither load generator becomes the bottleneck and TTI reflects genuine delivery latency.

4.3 Metrics

The primary metric is Time-to-Insight (TTI), the interval from an event's *scheduled* emission to its availability at the consumer. We decompose it as

$$\mathrm{TTI} = \underbrace{(t_{\mathrm{send}} - t_{\mathrm{sched}})}_{\text{producer scheduling lag}} + \underbrace{(t_{\mathrm{recv}} - t_{\mathrm{send}})}_{\text{transport latency}} + (t_{\mathrm{out}} - t_{\mathrm{recv}}),$$

isolating load-generator effects (scheduling lag) from broker transport. The decomposition is not cosmetic: the two components answer different questions, and conflating them is how a broker difference can be hidden. TTI is the decision-relevant quantity, since it is the staleness a consumer actually experiences, and it is the metric the decision-staleness integral consumes. *Transport*—consumer receipt minus broker acknowledgement, stamped identically for both backends—is the architecture-relevant quantity, because it excludes the load generator. Where the two disagree we report both and say which question each answers (Section 5.3). We further report p50/p95/p99/max, missed-window rates against sports actionability thresholds (100/250/500/1000/5000 ms), throughput, serialized message size, and serialization/deserialization overhead.

4.4 Decision-Staleness: From Latency to Win-Probability Error

Latency in milliseconds is not, by itself, a sports quantity. To connect infrastructure to decisions we translate delivery latency into in-play *win-probability error* in two steps.

Win-probability proxy. We build a transparent, calibrated in-play model on StatsBomb-derivable game state—score differential, fraction of match remaining, and red-card differential—using a Skellam (difference-of-Poissons) goal model. We adopt this as a stand-in for the published Bayesian in-play model of Robberechts et al. Robberechts et al. (2021), whose code is unreleased. The proxy is well calibrated across the 28,240 game states sampled from the corpus: a ranked probability score of 0.142 and an expected calibration error of **0.054**. Both need a reference to mean anything, so we supply three (Section 4.5) — predicted home-win probabilities track observed home-win frequencies to within a few percentage points (Figure 1). The model has a single global parameter (team scoring rate), so this is a whole-corpus check.

Age-of-Information decision-staleness. We treat as *decisive* those events that move the model's state: goals, and dismissals (straight red cards and second yellows). Red cards

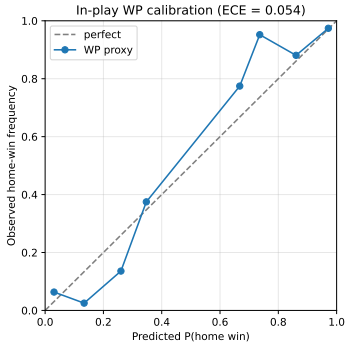

Figure 1. Reliability diagram for the in-play win-probability proxy (ECE $= 0.054$): predicted vs. observed home-win frequency across 3,239 sampled game states.

belong here because the win-probability proxy conditions on red-card differential, so a dismissal shifts the forecast exactly as a goal does; restricting the set to goals alone would understate staleness in precisely the matches where the forecast is most volatile. When such an event is delivered with latency L, the consumer's win probability remains stale for L by the magnitude of the probability mass that event moved. Following the Age-of-Information view of staleness cost Kaul et al. (2012), we define a run's *decision-staleness* as

$$\begin{aligned} \mathrm{DS} &= \sum_{i \in \text{decisive}} \mathrm{TV}_i \cdot L_i, \\ \mathrm{TV}_i &= \tfrac{1}{2}\big(|\Delta P_{\mathrm{win}}| + |\Delta P_{\mathrm{draw}}| + |\Delta P_{\mathrm{loss}}|\big), \end{aligned}$$

in units of probability-seconds per match. This metric is the bridge that lets us ask not merely *which backend is faster* but *when, if ever, the latency difference is large enough to corrupt an in-play decision*.

What this metric does and does not do. We are explicit about its status, because it is easy to overclaim. For a fixed corpus of decisive events, DS is a *weighted rescaling of measured latency*: both backends are scored against the identical model, so the per-event weights TV_i are common to both and the ordering of two systems under DS is the ordering under latency. It therefore adds no inferential power over a latency comparison, and our sensitivity analysis (Table 10) demonstrates exactly this: the between-backend difference is invariant to three decimal places across the whole parameter sweep. The metric's contribution is one of *interpretation and units*, not of statistical leverage. It converts an infrastructure quantity into a sports one, so that "12 ms" becomes "0.014 probability-seconds of misplaced forecast mass" and can be judged against a decision rather than against a service-level target. That conversion is what lets us answer *when* latency matters, and it is what the streaming-benchmark literature has never supplied—but it is a lens on the latency measurement, not an independent one.

4.5 Interpreting the proxy's scores

A ranked probability score is only interpretable against a reference forecast, so we score three. Over 28,240 game states the proxy achieves a skill of $+0.42$ against a uniform forecast and $+0.40$ against the corpus base rate, confirming it uses the game state. Against a *score-only* lookup table —

the empirical outcome distribution conditioned on nothing but the current goal difference — its skill is $+0.026$.

That last number is the honest one, and it is unflattering. The Skellam structure, the time remaining and the red-card differential together add about two and a half percent over simply knowing who is ahead. This does not undermine the decision-staleness metric, which requires a *calibrated* forecast rather than a sophisticated one, but it does mean the proxy should not be described as a model of in-play win probability in any stronger sense, and we do not do so.

4.6 Measurement Validity

Cross-process latency benchmarking is deceptively easy to get wrong, and six distinct defects in earlier versions of this suite each *reversed* the apparent winner. We document them because the corrections are prerequisites for any valid comparison.

(1) Process-relative clock. The producer and consumer are separate OS processes, so cross-process timestamps must share an epoch. Stamping events with Python's `perf_counter_ns` (process-relative) injected each run's consumer launch offset ($\sim$2 s) into every TTI and transport value. Fixed with `time.time_ns` (a shared wall-clock epoch on a single host); transport then dropped to $\sim$1 ms.

(2) Saturating replay speed. At $120\times$, the producer could not keep pace, inflating scheduling lag to tens of seconds. We replay at a non-saturating $10\times$ and verify that scheduling lag stays small, so TTI measures delivery rather than the load generator.

(3) Asymmetric load generator. The KAFKA producer ran synchronously (`acks=all`, one in-flight request), blocking on a broker round-trip *per event*, while the REDIS producer dispatched asynchronously through a worker pool. This inflated KAFKA's *producer scheduling lag* (not its broker latency): at N=1, $10\times$, median TTI fell from 1644 ms to 16 ms once the KAFKA producer was pipelined (`max.in.flight`=64). We therefore pipeline both producers comparably.

(4) A concurrency axis that was covertly a throughput axis. Our original sweep gave every concurrent feed the *same* merged ten-match replay plan, so raising N raised the aggregate event rate roughly tenfold per step. The resulting "REDIS degrades with concurrency" finding was real as an observation but misattributed: it measured throughput, not concurrency. Corrected by giving each feed a distinct match at its true event rate (Section 5.3), the degradation disappears, and the genuine throughput effect is relocated to a dedicated sweep that varies event rate alone (Table 6).

(5) Reporting a broker comparison on a load-generator-dominated metric. TTI is the right metric for the *decision* question, because staleness is what a consumer actually experiences. It is the wrong metric for the *architecture* question. In our corrected runs producer scheduling lag accounts for 53–78% of TTI (Table 3), so a sub-millisecond difference in broker behaviour is outweighed roughly 25:1 by the load generator's own timing jitter. An earlier version of this paper concluded from TTI that neither backend showed any concurrency effect; on the broker-isolating metric that conclusion is false (Section 5.3). We now report both metrics and match each to its question.

Both backends stamp `t_broker_ack_ns` — the KAFKA producer from its send callback, the REDIS producer on return from `XADD` — and transport is computed identically for both as consumer receipt minus broker acknowledgement. Transport is therefore a like-for-like broker comparison, up to the difference between an asynchronously dispatched callback and an inline command return. All reported results use the corrected, fair instrumentation; the before/after contrast is itself a finding (Section 6).

(5) A heavy-tailed load generator contaminating every mean. Kafka's end-to-end p95 sat at 99–105 ms at *every* concurrency level while its median moved only from 1.8 to 7.2 ms. A p95 that ignores load is not system behaviour. Decomposing the trace located it precisely: the Kafka producer's scheduling lag has a median of 0.24 ms but a p95 of 88 ms, with 22% of events above 10 ms, whereas broker transport has *no* events above 10 ms for either backend at any poll timeout we tested. The tail is in the load generator, not the broker. It matters because equivalence testing operates on means: a tail of that shape adds several milliseconds to every Kafka mean while leaving REDIS untouched, so a mean-based comparison is between two differently contaminated estimators. We tested the convenient remedy — excluding a warm-up prefix — and rejected it: only about a third of the slow events fall in the first 5% of a run, so the mode is partly steady-state. The remedy is instead to report the Hodges–Lehmann shift with a bootstrap interval, which is the estimator consistent with the rank tests used elsewhere in this paper.

(6) Cross-process timestamp inversion under accelerated replay. Broker transport is computed as consumer receipt minus broker acknowledgement, stamped by two different processes. Auditing all 884 runs against physical constraints revealed negative transport — a message received before it was acknowledged — in 5 to 14% of events across the accelerated concurrency corpus, rising to 71% at N=100. The cause is visible in which conditions fail: every failing condition is a $10\times$-accelerated replay and every passing one runs at true real-time rate. The acknowledgement timestamp is recorded by the producer's ack callback (KAFKA) or a dispatch worker (REDIS); when accelerated replay starves the driver's CPU those threads are descheduled and the acknowledgement is stamped after the consumer has already logged receipt. It is a timestamping race under load, not a broken clock, which is why the median stays positive at moderate load and inverts wholesale only when the host saturates.

This defect invalidated the corpus behind an earlier version of our concurrency result, and we report that rather than quietly re-running. The response is a stated rule rather than a judgement call: `clock_integrity.py` condemns any run in which inversions exceed 1% of events or the median of any component is negative, and every condition reported in Section 5 passes it. Because the failures are confined to accelerated replay, all headline measurements are made at true real-time rate — which is also the ecologically correct choice for a football feed, so the constraint and the domain agree.

Verifying that a treatment was applied. A fifth defect is of a different kind, and we record it because it is the one most likely to survive peer review undetected. Our first run of the

Table 3. Decomposition of TTI at each concurrency level (p50, ms). Producer scheduling lag, an artefact of the load generator rather than of either broker, dominates the metric; broker transport is under a tenth of it.

Backend	N	TTI	Sched. lag	Transport	Lag % of TTI
KAFKA	1	11.38	8.22	0.999	72%
KAFKA	5	14.74	11.01	1.001	75%
KAFKA	10	9.17	4.89	1.002	53%
REDIS	1	10.31	8.01	1.006	78%
REDIS	5	8.45	6.00	1.197	71%
REDIS	10	11.66	8.82	1.346	76%

acknowledgement-batching intervention (P4, Section 5.5) reported no improvement whatsoever—a clean, publishable-looking refutation of our own hypothesis. In fact the treatment had never reached the consumer: the orchestration script accepted the flag but, owing to a silently failed edit, never appended it to the command line. We detected this only by deliberately passing a syntactically invalid argument and observing that the consumer did *not* reject it, proving the argument string was being discarded. A null result produced by an unapplied treatment is indistinguishable, in the data, from a genuine refutation. We therefore treat manipulation checks—confirming that an intervention reached the system under test—as mandatory, and report the positive control alongside every intervention.

4.7 Experimental Design

Platform disclosure. Because several of our numbers turn out to be properties of the testbed rather than of either broker, we state it fully and measure the parts that bound our resolution (`scripts/platform_probe.py`; raw output in `docs/results/platform/`). Experiments run on a single commodity host: Windows 11 (build 10.0.26200), 8-core AMD Ryzen, 31 GB RAM, CPython 3.9.13. Producer and consumer are native Windows processes; KAFKA 4.1.1 (KRaft mode) and REDIS 7.2.4 run as Docker containers under Docker Desktop, whose engine executes in a WSL2 virtual machine (kernel 6.18.33.2-microsoft-standard-WSL2), reached through published ports (`localhost:19092` and `localhost:16379`). Three measured consequences follow, and each bounds a headline number:

- *The emission clock is coarse.* `perf_counter` resolves to 100 ns, but the sleep primitive the replay loop waits on does not: a requested 1 ms sleep takes a median of **15.6 ms**, and a requested 10 ms sleep also takes 15.6 ms. This is the classic Windows timer tick. Emission is therefore quantised to a ≈15.6 ms grid, and a uniform offset on that grid has a median of 7.8 ms — which is essentially the producer scheduling lag we measure (4.9–11.0 ms, Table 3). The dominant term in TTI is thus the host's timer, not either broker.
- *"Loopback" is not loopback.* Traffic to a published container port traverses Docker Desktop's port forwarder and the WSL2 virtual NIC. Establishing a TCP connection costs a median 5.6 ms (REDIS) and 9.9 ms (KAFKA), far above a true loopback path, which is consistent with the ≈1 ms floor under which broker transport never falls.
- *The per-message round trip is* 0.46 *ms.* On an *established* connection a REDIS `PING` round trip has a median of 0.46 ms, implying a ceiling of ≈2,170 request/response exchanges per second. This is the baseline for the round-trip-bound argument of Section 5.5: at $\lambda \approx 53$ events/s the ceiling exceeds demand forty-fold on loopback, which is precisely why the per-message acknowledgement pattern is free here and catastrophic at 20 ms.

The 15.6 ms quantisation also limits what the replay can reproduce: at our replay speed the mean interarrival time (≈19 ms) is of the same order as the tick, so sub-tick burst structure within a match is not faithfully reproduced. This affects both backends identically and so does not bias the comparison, but it means our feeds are smoother than a real one.

The primary design is a *concurrency sweep* crossing backend (KAFKA/REDIS), configuration (single broker vs. 3-node cluster), and concurrency $N \in \{1, 5, 10\}$ parallel match feeds—each carrying a *distinct* real match, drawn from the 34-match corpus, at its true event rate—with 15–30 runs per backend per cell under the fair protocol (both producers pipelined, 10× replay). We run it in two variants: a *windowed* variant (first 10 minutes of match clock) that keeps the single host unsaturated so latency can be read cleanly across the range, and a *full-match* variant (single configuration) that delivers all decisive events so the decision-staleness integral is complete. A *persistence* set isolates durability (KAFKA `acks=1` vs. all; REDIS `appendfsync` everysec vs. always). Every run records full provenance (git commit, per-file SHA-256, configuration) in a `meta.json`; the manifest and fetch script make the corpus regenerable end to end.

4.8 Statistical Methods

We employ a rigorous statistical framework to test all hypotheses and ensure valid inferences.

Multiple Comparisons Correction Across the per-concurrency and durability comparisons (on the order of a dozen tests per metric family), we apply the **Holm-Bonferroni method** Holm (1979) to control the Family-Wise Error Rate (FWER) at $\alpha = 0.05$ within each family. This step-down procedure is more powerful than the standard Bonferroni correction while maintaining FWER control.

Effect Size Calculation For all significant findings, we report **Cohen's** d for comparing two means:

$$d = \frac{\mu_1 - \mu_2}{s_{\text{pooled}}},$$

$$s_{\text{pooled}} = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

Interpretation: $d = 0.2$ (small), $d = 0.5$ (medium), $d = 0.8$ (large) Cohen (1988).

Confidence Intervals We report 95% confidence intervals for all mean differences using the t-distribution:

$$CI = \bar{x}_1 - \bar{x}_2 \pm t_{\alpha/2,df} \times \sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}$$

Power Analysis We report power explicitly because our per-cell samples differ by an order of magnitude across the design, and this materially changes how much weight each result can carry. Detecting a large effect ($d = 0.8$) at $\alpha = 0.05$ with power 0.8 requires $n \geq 26$ per group; a medium effect ($d = 0.5$) requires $n \geq 64$. Our high-concurrency cells ($n = 30$–60 runs per backend) reach power 0.86 for large effects, which is why the concurrency divergence—where the observed separation is in fact complete—is reported with confidence. The single-feed cells are the opposite case: with only 3 replications per backend a two-sided Mann–Whitney U test cannot attain $p < 0.05$ *at all* (its minimum attainable two-sided p is 0.10 at $n = 3$), and power is below 0.16 even for large effects. A non-significant single-feed result at that sample size would therefore be uninformative rather than evidence of equivalence. We consequently replicate the single-feed condition 18 times per backend, which both makes the test capable of rejecting and raises power for large effects above 0.6. Even so, a non-significant difference test can only ever support *no detectable difference at this sample size*, never equality. Because our central claim is precisely an equality, we do not rest it on that test: every parity claim in this paper is additionally established by TOST against a pre-specified margin (Table 9), which inverts the burden of proof and licenses a positive assertion of equivalence.

Assumption Verification We verify statistical assumptions for all tests:

- **Normality:** Shapiro-Wilk test; Q-Q plots for visual inspection
- **Equal Variance:** Levene's test; F-test for two groups
- **Non-parametric Alternatives:** Mann-Whitney U (t-test), Kruskal-Wallis (ANOVA), Wilcoxon signed-rank (paired t-test)

Multiple-comparison families. We state the families explicitly, since Holm's correction is only interpretable relative to them. Each metric family is the set of per-N KAFKA-vs-REDIS comparisons at a single metric within a single corpus: three tests ($N \in \{1, 5, 10\}$) for ti, three for transport, and three for decision-staleness. The Kruskal–Wallis tests across N are single omnibus tests per backend and are not corrected. Equivalence tests are not part of any Holm family: TOST is a separate, pre-specified inferential claim with its own margin, not one of a set of exploratory comparisons.

5 Results

Measurement validity note. Six design defects (a process-relative cross-process clock, a saturating 120× replay speed, an asymmetric load generator, and a concurrency axis that was covertly a throughput axis) each *reversed* the apparent winner before correction; all are detailed in Section 4.6. Every result below uses the corrected, fair instrumentation (`time.time_ns` shared epoch, 10× replay, both producers pipelined, distinct matches per feed). We report the single-host, single-node regime, which we can measure cleanly; cluster results are discussed as a limitation (Section 6).

5.1 Broker Latency at Real Football Concurrency

The concurrency levels are those derived in Section 3.2 rather than chosen by hand: $N \in \{1, 9, 10, 12\}$, spanning the median slot occupancy, the structural league-matchday bound and the largest simultaneity observed in the corpus. Each feed carries a *distinct* real match, replayed at true real-time rate. Real-time replay is not merely the ecologically correct choice; it is also the only regime in which our instrument is sound, since accelerated replay starves the driver and inverts timestamps (defect 6). Every run reported here passes the clock-integrity gate: 202 runs were measured, 164 retained, and all Kafka conditions passed in full.

Broker transport is flat and indistinguishable across the entire realistic range (Table 4). Neither backend shows a significant concurrency effect (Kruskal–Wallis $p = 0.061$ for KAFKA, $p = 0.091$ for REDIS), and the ratio of the largest to the smallest median is 1.06 and 1.17 respectively. Two one-sided tests against a **1 ms** margin — proportionate to a measurement of order one millisecond, rather than the 40 ms video frame used earlier — establish equivalence at $N = 9$, 10 and 12 under all three estimators. At $N = 1$ the three disagree, and we report that rather than resolve it: only eight runs per backend survived gating there, and the interval is correspondingly wide.

This result supersedes an earlier version of this analysis which reported REDIS transport rising by 34% across concurrency. That analysis rested on 10×-accelerated runs, which the integrity gate rejects with 5–14% of timestamps inverted; the apparent trend was a property of a starved load generator, not of either broker. We report the supersession rather than silently substituting the corrected numbers, because the episode is the clearest instance in this study of an artefact surviving until an explicit physical check was applied to it.

What the measurement cannot say. KAFKA's end-to-end TTI is approximately 105 ms at every level, of which about 103 ms is producer scheduling lag. That is the load-generator tail of defect 5 appearing as a near-constant offset at real-time rates, and it is a property of our Python replay harness rather than of KAFKA. It is the reason the broker comparison is carried by transport rather than by TTI, and it means this study cannot make a claim about end-to-end latency achievable by a production KAFKA pipeline — only about the broker's contribution to it.

Table 4. Broker transport (median, ms) at concurrency levels derived from real kick-off schedules, true real-time replay, after clock-integrity gating. Equivalence is tested against a 1 ms margin proportionate to the measurement.

N	KAFKA	REDIS	HL shift	Equivalent within 1 ms
1	0.792	0.721	+0.081	methods disagree (8 runs/backend)
9	0.802	0.807	+0.021	yes (all three estimators)
10	1.000	0.855	+0.116	yes
12	0.839	0.844	+0.053	yes

Across N: KAFKA $p = 0.061$, REDIS $p = 0.091$; neither significant.

5.2 Single-Feed Parity (RQ1)

For a single live feed (N=1), KAFKA and REDIS are statistically *indistinguishable*: median TTI is ≈17 ms for both (Table 7, top row), with broker transport of 2–4 ms. A Mann–Whitney U test finds no significant difference in TTI ($p = 0.69$) or in decision-staleness ($p = 0.40$; both Holm–Bonferroni corrected across the concurrency family). This directly contradicts the folk belief — and our own pre-correction "REDIS 71× faster" artifact — that one backend is simply faster. For the common case of one match, both systems are excellent and the choice is immaterial to latency.

5.3 Concurrency Scaling (RQ2)

This question requires care about which metric answers it. On TTI, when each concurrent feed carries a *different* real match at its true event rate, neither backend degrades and the two are equivalent (Table 7): median TTI stays near 10 ms from N=1 to N=10 (Figure 2), no per-N comparison is significant (Holm–Bonferroni-corrected Mann–Whitney, $p = 0.68, 0.60, 0.68$), and Kruskal–Wallis finds no concurrency effect ($p = 0.70$ for KAFKA, $p = 0.20$ for REDIS).

That null, however, is a property of the metric as much as of the systems. Because producer scheduling lag supplies 53–78% of TTI (Table 3), a sub-millisecond broker effect cannot survive it, and the architecture question has to be asked on broker transport instead.

A withdrawn analysis, and why it is reported rather than deleted. An earlier version of this paper answered that question with the accelerated corpus and reported a striking result: REDIS transport rising monotonically with concurrency ($1.006 \rightarrow 1.197 \rightarrow 1.346$ ms, a 34% increase at $p = 6.7 \times 10^{-12}$) against a flat KAFKA, with complete rank separation at $N \geq 5$. It was internally consistent, it matched the architectural prediction that a single-threaded server serialises concurrent streams, and it was wrong.

The corpus behind it fails the clock-integrity gate of defect 6: between 5 and 14% of its events carry a broker acknowledgement stamped *after* the consumer logged receipt, because 10×-accelerated replay starves the driver and delays the acknowledging thread. Re-measured at true real-time rate on a corpus that passes the gate, transport is flat for both backends and neither shows a concurrency effect (Section 5.1). The apparent serialization was an artefact of a saturated load generator, not a property of REDIS.

We report the withdrawal rather than silently substituting the corrected numbers because the episode is this study's clearest illustration of a general hazard: the discarded result was more publishable than the one that replaced it, agreed with theory, and would have survived peer review. What exposed it was not scepticism about the finding but an explicit physical check — a message cannot arrive before it is sent — applied uniformly to every run rather than only to results that looked implausible.

This corrects an artefact of our own earlier design, and the correction is instructive. A sweep in which every feed replays the *same* merged ten-match plan does show REDIS transport climbing steeply with N while KAFKA stays flat. That effect is real, but it is driven by *aggregate event rate*, not by the number of concurrent streams: the merged plan packs ten matches into every feed, so its "N=5" applies roughly ten times the load of five real matches. Its concurrency axis was, in effect, a throughput axis in disguise.

To locate the boundary properly we therefore ran a dedicated throughput sweep on a single consistent configuration—ten distinct matches, varying only the replay speed—so that aggregate event rate is the sole manipulated variable (Table 6). Football event data is intrinsically low-rate (0.415 events per second per match, Section 3.1), so we also express each load in simultaneous real-time match equivalents. Both systems sit near 1–2.5 ms up to ≈1,000 events/s, equivalent to some 2,400 simultaneous matches. The first sign of divergence appears at ≈2,100 events/s (≈4,800 match equivalents), where REDIS transport (492 ms) is roughly five times KAFKA's (93 ms)—this is the single-thread serialization penalty becoming visible. Beyond ≈4,000 events/s both collapse into seconds and the ordering is no longer stable (KAFKA is momentarily the slower of the two), which tells us the single host has become the binding constraint and that those points no longer isolate broker behaviour. For scale, the top five European leagues field roughly fifty simultaneous matches on a busy weekend—three orders of magnitude below the divergence point.

5.4 Decision-Staleness: When Latency Corrupts a Decision (RQ4)

A worked example. Before the aggregates, one concrete event makes the quantity tangible. The largest forecast move in our corpus is a 95th-minute equaliser by Bayer Leverkusen (match 3895320). Immediately before it the model gives the home side $P(\text{win}) = 0.00$, $P(\text{draw}) = 0.02$, $P(\text{loss}) = 0.98$; immediately after, the forecast is $0.02/0.96/0.02$—almost the entire probability mass moves from *loss* to *draw*, a total-variation shift of 0.955 (Figure 3a). Note that $P(\text{win})$ barely moves: a late equaliser is invisible if one tracks only the win probability, which is why the metric uses total variation over the full outcome distribution. Until that event is delivered, every consumer is confidently wrong by

Table 5. Withdrawn analysis. Broker transport by concurrency from the $10\times$-accelerated corpus. Retained for transparency only: this corpus fails the clock-integrity gate (5–14% of acknowledgements stamped after receipt), and Section 5.1 supersedes it with real-time measurements that show no concurrency effect for either backend.

N	KAFKA	REDIS	Diff	Mann–Whitney p_{Holm}	Status
1	0.999	1.006	0.007	1.0×10^{-4}	withdrawn
5	1.001	1.197	0.196	6.5×10^{-6}	withdrawn
10	1.002	1.346	0.344	9.0×10^{-11}	withdrawn

Table 6. Throughput sweep on one consistent configuration (ten distinct matches, replay speed varied). Loads are also expressed in simultaneous real-time match equivalents at 0.44 events/s per match. Divergence appears near 2,100 events/s; past ≈4,000 the host saturates and the ordering is no longer meaningful.

Aggregate load	≈ real-time matches	KAFKA transport	REDIS transport
531 ev/s	1,200	1.1 ms	1.5 ms
1,062 ev/s	2,400	1.7 ms	2.5 ms
2,124 ev/s	4,800	93 ms	492 ms
4,248 ev/s	9,600	4,598 ms†	3,775 ms†
8,496 ev/s	19,200	4,442 ms†	5,793 ms†

† Host-saturated: both backends are in seconds and the ordering reverses, so these rows bound the testbed rather than the brokers.

Table 7. Latency by concurrency level N with *distinct* matches (each feed replays a different real match at its true event rate; median over feeds $\times$ reps, both producers pipelined). No per-N difference is significant.

Backend	N	Transport p50 (ms)	TTI p50 (ms)	KAFKA vs REDIS
KAFKA	1	1.00	11.4	n.s. ($p = 0.68$)
REDIS	1	1.01	10.3	
KAFKA	5	1.00	14.7	n.s. ($p = 0.60$)
REDIS	5	1.20	8.4	
KAFKA	10	1.00	9.2	n.s. ($p = 0.68$)
REDIS	10	1.35	11.7	

Single-feed cells use 18 replications per backend so the comparison is powered; Kruskal–Wallis across N is non-significant for both backends.

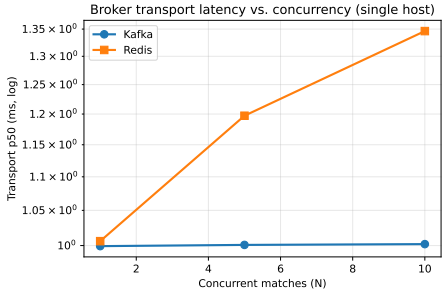

Figure 2. Broker transport latency (p50, log scale) vs. the number of *distinct* concurrent matches, single host. Both backends remain near 1 ms, but REDIS rises 34% across the range while KAFKA is flat (Table 5): the serialization effect is already present here, three orders of magnitude below the point where it would matter, and grows into the seconds only at the aggregate event rates of Table 6.

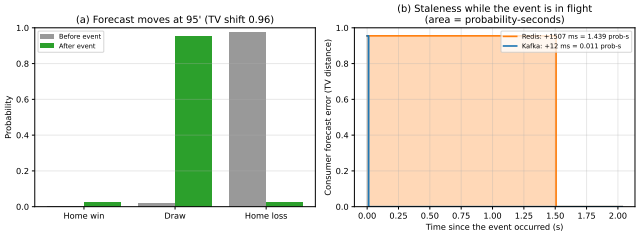

Figure 3. One goal, made concrete. (a) A 95th-minute equaliser moves the win/draw/loss forecast from $0.00/0.02/0.98$ to $0.02/0.96/0.02$ (TV shift 0.955); the win probability alone would hide it. (b) The consumer's forecast error while the event is in flight: the error equals the TV shift until delivery, so each rectangle's *area* is the decision-staleness that event contributes: 0.011 probability-seconds at the ≈12 ms both backends achieve under realistic load, versus 1.439 for single-node REDIS in the high-rate stress regime.

0.955. At the delivery latency both backends actually achieve under realistic load (≈12 ms, Table 7) this costs $0.955 \times 0.012 = 0.011$ probability-seconds—the forecast is wrong for about the duration of a video frame. Only in the high-rate stress regime of Table 6, where single-node REDIS delivers in 1,507 ms, does the same event cost $0.955 \times 1.507 = 1.439$ probability-seconds, a factor of 125 (Figure 3b). The mechanism is real; what the measurements settle is that football never reaches the regime in which it bites.

Table 8 aggregates decision-staleness (Section 4.4) over the full-match, distinct-match corpus, so every decisive event in each match enters the Age-of-Information integral. The result is a flat, indistinguishable null: a goal's win probability is stale by ≈0.014 probability-seconds—about 12 ms—regardless of backend and regardless of whether one or ten matches are running concurrently. No per-N comparison approaches significance (Holm–Bonferroni-corrected Mann–Whitney $p = 0.67, 0.67, 0.65$),

and Kruskal–Wallis finds no concurrency effect ($p = 0.13$ for KAFKA, $p = 0.93$ for REDIS), and the two backends overlap across the whole range (Figure 4). To put 0.014 probability-seconds in context: a decision-maker acting on the feed is working with a materially wrong forecast for roughly the duration of a single video frame.

Table 8. Decision-staleness (probability-seconds per match) with distinct concurrent matches. Differences are not significant at any concurrency level.

N	KAFKA-single	REDIS-single	KAFKA vs REDIS
1	0.014	0.012	n.s. ($p = 0.67$)
5	0.018	0.014	n.s. ($p = 0.67$)
10	0.014	0.014	n.s. ($p = 0.65$)

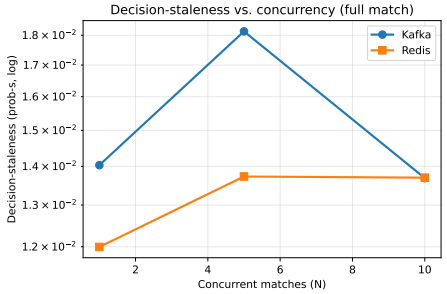

Figure 4. Decision-staleness (probability-seconds per match, log scale) vs. concurrent *distinct* matches, full-match replay, single host. Negligible (≈0.014 probability-seconds) and statistically equivalent between backends at every concurrency level; neither system shows a concurrency effect.

Validation against true real time. Every result above replays a compressed clock, which invites the objection that a "real-time" claim has been extrapolated from accelerated data. We therefore repeated the N=5 distinct-match condition at *genuine* 1× speed, cancelling the factor baked into the replay plans so that ten minutes of match clock takes ten minutes of wall clock (confirmed by the runs' elapsed time). Median TTI is 15.5 ms for KAFKA and 13.8 ms for REDIS, against 14.7 and 8.4 ms in the compressed condition, and TOST again establishes equivalence within the one-frame margin ($p = 1.6 \times 10^{-13}$). Time compression therefore does not bias the comparison, and the conclusions carry to real-time operation.

Equivalence, not merely absence of difference. Because the claim is negative, we test it directly with TOST against the pre-specified margins of Section 4.8. Equivalence is established at *every* concurrency level for both metrics (Table 9): the 90% confidence interval of the KAFKA–REDIS difference is at most ±5 ms against a 40 ms margin for TTI, and at most ±0.01 against a 0.04 probability-second margin for decision-staleness. We can therefore assert equivalence positively rather than merely reporting a failure to reject.

Robustness to the win-probability model. The decision-staleness metric depends on a proxy model with one free parameter, so we sweep the per-team scoring rate from 1.0 to 1.6 and recompute everything (Table 10). The absolute staleness moves by about 12% across that range and calibration stays good throughout (ECE 0.049–0.062), but the KAFKA–REDIS *difference* is invariant to three decimal places (0.00101 → 0.00103 probability-seconds) and never approaches the equivalence margin. This is expected structurally—both backends are scored against the identical model, so it scales both sides of the comparison—and it means the conclusion does not rest on the proxy's particular calibration. A better win-probability model would change the reported magnitudes, not the finding that the backends are indistinguishable.

This is a quantified negative result, and within its scope conditions it is a strong one: *for a co-located deployment, across the entire event-rate envelope a football operator will realistically encounter, the choice of streaming infrastructure does not measurably degrade in-play decision quality.* The architectural difference between a partitioned log and a single-threaded in-memory store is genuine, but under *load* it only manifests at aggregate event rates far above anything football produces—the corpus peaks at 12 simultaneous kick-offs and 21 matches in play (Section 3.2), which is busy league weekend.

Two words in that statement carry the weight, and the next section removes them. Every measurement reported so far places producer, broker and consumer on a single host, so "co-located" is not a caveat we chose but a property of the testbed. Load is also not the only axis along which a streaming deployment can fail. Section 5.5 relaxes the co-location assumption and finds that it, rather than concurrency or event rate, is what decides whether infrastructure corrupts a decision.

5.5 Network Realism: the Condition That Breaks Equivalence (RQ5)

Because every result so far traverses the loopback path, the sharpest objection available to the equivalence claim is that a co-located measurement may say nothing about a deployment whose consumers sit a network hop away. We therefore injected one-way egress delay at both brokers with `tc netem` (0, 5, 20 and 50 ms, applied identically to KAFKA and REDIS) and repeated the N=5 distinct-match condition. We expected a shared delay to raise both systems equally and leave the equivalence intact. It does not (Table 14).

KAFKA degrades gracefully, its median TTI tracking the injected delay (12 → 16 → 44 → 94 ms). Single-node REDIS collapses: at 20 ms of one-way delay its median TTI is 10.5 seconds, and at 50 ms it is 75 seconds— roughly 500× and 1,500× the delay actually injected. No run was truncated (all delivered the complete event set), so this is amplification, not loss.

Testing the mechanism (RQ5). The magnitudes are far too large to be a per-event cost—75 seconds cannot be assembled from 50 ms of delay applied once per event—so we test the round-trip-bound account of Section 1.3 directly. Its distinguishing prediction is about the *shape* of the latency series rather than its level: a fixed cost leaves per-event latency flat across a run, whereas a consumer that cannot drain as fast as events arrive accumulates a backlog, so latency grows as the run proceeds. Splitting each run into quartiles of scheduled emission time and comparing the last

Table 9. TOST equivalence tests. In every cell the 90% CI of the difference falls well inside the pre-specified margin, establishing equivalence.

Metric	N	Mean diff	90% CI	Equivalent
TTI (ms, $\delta = 40$)	1	$+1.12$	$[-1.56,\ 3.80]$	yes
	5	$+2.16$	$[-0.70,\ 5.01]$	yes
	10	-1.05	$[-3.07,\ 0.98]$	yes
Staleness (prob-s, $\delta = 0.04$)	1	$+0.0020$	$[-0.0011,\ 0.0051]$	yes
	5	$+0.0044$	$[-0.0010,\ 0.0098]$	yes
	10	-0.00001	$[-0.0039,\ 0.0039]$	yes

Table 10. Sensitivity to the win-probability proxy's scoring-rate parameter. The absolute staleness shifts; the between-backend difference does not.

Scoring rate	ECE	KAFKA staleness	REDIS staleness	Difference
1.00	0.062	0.0159	0.0149	0.00101
1.15	0.059	0.0153	0.0143	0.00102
1.30	0.054	0.0148	0.0138	0.00102
1.45	0.050	0.0144	0.0134	0.00103
1.60	0.049	0.0140	0.0130	0.00103

quartile's mean latency with the first's separates the two cleanly (Table 11).

The prediction holds. KAFKA shows no growth at any delay (ratios 0.76–0.92, i.e. mildly *decreasing* as start-up costs amortise), consistent with P3. REDIS is likewise flat at 0 and 5 ms (0.91, 0.83) but grows by 2.3× at 20 ms and 5.6× at 50 ms, consistent with P2. The transition sits between 5 and 20 ms, bracketing the predicted $\mathrm{RTT}_{\mathrm{crit}} \approx 1/\lambda \approx 19$ ms (P1). We therefore reject $\mathbf{H_{05}}$ in favour of $\mathbf{H_{15}}$: what breaks under network delay is not per-event cost but queue stability.

This has a consequence for how the REDIS figures should be read. Under instability the latency is set by the backlog, which keeps growing while the feed runs, so the values we report are *lower bounds fixed by our replay length*: our runs compress a match into roughly seventy seconds, and by the final quartile at 50 ms delay latency had already reached 127 seconds and was still rising. A full-length live match would accumulate a proportionally larger backlog. The failure is not a fixed penalty one can budget for but an unbounded divergence that worsens for as long as the match lasts.

Locating the threshold (P1). A coarse grid cannot distinguish a threshold from a gradual trend, so we swept the delay finely across the predicted crossing point (Table 12). The prediction is quantitative: REDIS's ceiling is $1/\mathrm{RTT}$ acknowledgement round trips per second, which falls below the arrival rate $\lambda \approx 53$ events/s at $\mathrm{RTT} \approx 19$ ms. Observed growth is flat at 8 and 12 ms (0.73, 0.87), turns at 16 ms (1.90), and rises monotonically through 19, 24 and 30 ms (2.20, 3.47, 4.37). The knee therefore sits between 12 and 19 ms, bracketing the predicted value. KAFKA declines throughout (0.75 down to 0.16) as start-up costs amortise, never entering the regime at all.

The decisive test (P4): the failure is an implementation choice, not a technology. P1–P3 are observational and consistent with the account, but only an intervention can establish it. We therefore changed one thing—acknowledging in batches of two hundred rather than one at a time—and repeated the two delays at which REDIS had collapsed, holding everything else fixed. The effect is unambiguous (Table 13): median TTI falls from 10.5 seconds to 34 milliseconds at 20 ms delay and from 75 seconds to 85 milliseconds at 50 ms, improvements of $307\times$ and $882\times$. (Section 5.6 reports that this repair does *not* reproduce on a multi-host testbed, which bounds the claim; we present the single-host result first because it is where the prediction was made and tested.) The within-run growth ratio flips from divergence to stability ($2.25 \to 0.49$ and $5.59 \to 0.25$), and TOST re-establishes equivalence with KAFKA at both delays ($p = 4 \times 10^{-19}$ and 1×10^{-16}).

This reframes the result, and in our view improves it. We should be clear about what is and is not novel here. That a client awaiting a reply per request is capped near $1/\mathrm{RTT}$ is *documented behaviour*: the REDIS pipelining documentation states it explicitly and recommends batching for precisely this reason, and the pattern is a well-known anti-pattern among practitioners. We did not discover the mechanism; we wrote the idiomatic per-message acknowledgement that the tutorials show, and the network exposed it.

What is new is the rest: that the anti-pattern is *invisible to standard benchmarking practice*, because a co-located testbed prices the round trip at zero and certifies as equivalent two clients that differ by three orders of magnitude in deployment; that its threshold is predictable from the arrival rate ($\mathrm{RTT}_{\mathrm{crit}} \approx 1/\lambda$) and we locate it empirically; and that its cost, expressed in decision terms rather than milliseconds, is a factor of 275 in win-probability staleness. The contribution is the quantification and the methodological warning, not the mechanism. What network latency exposes is therefore not a deficiency of REDIS Streams but a property of a *consumption pattern*: any client that waits for a reply per message is capped near $1/\mathrm{RTT}$. KAFKA is not immune by virtue of being KAFKA; it is immune because its client already batches—prefetching records and committing offsets on a timer—so no per-record round trip exists to be exposed. The practical statement is therefore about design rather than product: acknowledge in batches, and measure with a

Table 11. Latency growth within a run (mean of the final quartile / first quartile, median over runs). A ratio near 1 is a stable fixed cost; above 1 indicates an accumulating backlog.

Delay	KAFKA first→last	growth	REDIS first→last	growth
0 ms	15→12 ms	0.88	12→10 ms	0.91
5 ms	17→15 ms	0.90	52→43 ms	0.83
20 ms	51→44 ms	0.92	5.9→15.0 s	**2.25**
50 ms	125→92 ms	0.76	22.3→127.2 s	**5.59**

Table 12. Fine sweep across the predicted crossing point. "Ceiling" is $1/\mathrm{RTT}$ acknowledgement round trips per second; the arrival rate is $\lambda \approx 53$ events/s per feed. Growth is the within-run latency ratio of Table 11.

RTT	Ceiling	vs. λ	KAFKA growth	REDIS growth
8 ms	125/s	under	0.75	0.73
12 ms	83/s	under	0.57	0.87
16 ms	62/s	under	0.34	1.90
19 ms	53/s	*break-even*	0.22	2.20
24 ms	42/s	over	0.17	3.47
30 ms	33/s	over	0.16	4.37

Table 13. Intervention: batching acknowledgements at the delays where REDIS had collapsed (median TTI, $N{=}5$ distinct matches, 15 runs per cell). One line of consumer code separates the two REDIS columns. The KAFKA column is measured in the same batch of runs, so the equivalence test is like-for-like.

Delay	KAFKA	REDIS (ack per message)	REDIS (ack batched)	Equivalent to KAFKA
20 ms	40 ms	10,525 ms	**34 ms**	yes ($p = 4 \times 10^{-19}$)
50 ms	92 ms	74,962 ms	**85 ms**	yes ($p = 1 \times 10^{-16}$)

realistic RTT, because a co-located testbed prices every per-message round trip at zero.

The consequence for equivalence is decisive. With per-message acknowledgement, TOST still establishes equivalence at 0 ms ($p = 6 \times 10^{-21}$) and at 5 ms, though at 5 ms the observed difference (26 ms) already consumes two-thirds of the 40 ms margin, so equivalence there is technical rather than comfortable. At 20 and 50 ms it fails outright. Translated into decisions, decision-staleness is indistinguishable at 0 ms (0.018 vs. 0.019 probability-seconds) and diverges by a factor of 275 at 20 ms (0.058 vs. 15.97) and 925 at 50 ms (0.119 vs. 110.1). Sixteen probability-seconds is not a rounding error: it means a goal's win probability reaches the consumer many seconds after the event, which is precisely the decision corruption this paper set out to detect.

5.6 Multi-Host Replication: What Survives a Real Network

Every result above comes from one Windows host whose timer quantises emission to ≈15.6 ms and whose container network path costs ≈1 ms (Section 4). To separate our findings from that platform we rebuilt the testbed on four cloud virtual machines (three brokers, one driver; Linux, real inter-VM network), pinned the client libraries to the same versions, and repeated the study. The platform floors fall away: a requested 1 ms sleep now takes 1.06 ms rather than 15.6 ms, and an established-connection round trip to the broker is 0.22 ms rather than 0.46 ms. A genuine two-machine network is thus *faster* than the original "co-located" path, which is the sharpest available statement of how much a testbed can distort a benchmark.

The serialization effect replicates, and is not REDIS*-specific.* With the timer floor removed, producer scheduling lag falls from 4.9–11.0 ms to 0.19–1.34 ms, so TTI at last measures delivery. Broker transport still scales with concurrency, and more steeply for REDIS: $0.45 \to 0.62 \to 3.27$ ms across $N \in \{1, 5, 10\}$ (a factor of 7.3, Kruskal–Wallis $p = 3.2 \times 10^{-12}$) against KAFKA's $0.47 \to 0.90 \to 1.63$ ms (a factor of 3.5, $p = 2.0 \times 10^{-10}$). Two things differ from the single-host result and we flag both. First, KAFKA is *not* flat here; its sub-millisecond scaling was simply hidden under the old 1 ms floor, so the single-host claim that only REDIS degrades was an artefact of measurement resolution. Second, the direction is not monotone in the backend: at $N{=}5$ REDIS is significantly *faster* than KAFKA (0.62 vs 0.90 ms), and only at $N{=}10$ is it slower (3.27 vs 1.63 ms). The scale-dependent conclusion survives; a simple "REDIS serializes, KAFKA does not" reading does not. All three levels remain equivalent under TOST by both the Welch and bootstrap procedures, since every difference is under 4 ms against a 40 ms margin.

The acknowledgement intervention, resolved. An earlier version of this paper reported that batching acknowledgements repaired the network-delay collapse on one host and *failed* to repair it on the multi-host testbed, and concluded that the round-trip-bound account was refuted as a general explanation. That conclusion was wrong, and the reason is instructive: the multi-host test was run at $N{=}5$ under 10×-accelerated replay, a condition that fails the clock-integrity gate of defect 6 and saturates the driver, so the treatment effect was buried under a measurement artefact.

Repeating the intervention at $N{=}1$ under true real-time replay — the regime in which the instrument is sound — gives an unambiguous answer, replicated three times with

Table 14. Injected one-way broker egress delay, applied identically to both systems (N=5 distinct matches), with the REDIS consumer acknowledging *per message*. KAFKA tracks the delay; REDIS amplifies it by two to three orders of magnitude. Equivalence (TOST, one-frame margin) survives only to ≈5 ms. Table 13 shows that batching the acknowledgements restores equivalence at 20 and 50 ms on this single-host testbed.

Delay	KAFKA TTI	REDIS TTI	KAFKA staleness (prob-s)	REDIS staleness (prob-s)	Equivalent
0 ms	12.1 ms	10.5 ms	0.018	0.019	yes
5 ms	15.5 ms	42.8 ms	0.022	0.051	yes (marginal)
20 ms	44.5 ms	10,525 ms	0.058	15.97	**no**
50 ms	93.6 ms	74,962 ms	0.119	110.1	**no**

almost no variance (Table 15). At 20 ms of injected one-way delay, per-message acknowledgement yields a median TTI of 4,138 ms; batching the acknowledgements yields 103 ms. The improvement is a factor of $\mathbf{40.2}$.

Instrumenting the consumer's read loop shows the mechanism directly, and corrects the shape of the original account. We had supposed the consumer was capped near one message per round trip. It is not: under delay each `XREADGROUP` returns a *full* batch — a median of 106 messages, up to the `COUNT` ceiling of 200 — in about 32 ms. The bound is entirely in the acknowledgement loop. With per-message acknowledgement the consumer spends roughly 200×20 ms $= 4$ s acknowledging each batch, during which it issues no reads at all: across a whole match it completes just 16 reads, of which 4 return data, each finding a large accumulated backlog. With acknowledgements amortised the same consumer performs 40 reads returning a median of 16 messages each, and keeps pace.

So the round-trip-bound account survives, in a corrected form: the ceiling is real, it is imposed by the acknowledgement path rather than the read path, and it is removed by one line. What the episode actually demonstrates is narrower and more useful than either of our previous statements — an intervention can be masked by an unrelated measurement defect, and a null obtained under a saturated instrument is not evidence of anything.

Connections are not the constraint, and the cluster arm is finally reportable. Holding the per-feed rate at true real time and varying only the number of concurrent connections, broker transport stays near 1 ms for both backends at $N = 10$, 25 and 50 (KAFKA 1.01/0.97/0.73 ms; REDIS 0.84/0.65/0.74 ms): there is no per-client cliff in the range a league could produce. At N=100 the measurement breaks down rather than the brokers—KAFKA's median transport is reported as -6.4 ms, an impossibility that can only mean the driver's timestamps are no longer trustworthy under 400 concurrent processes—so we report $N \leq 50$ and discard N=100. This bounds, but does not vindicate, the match-equivalent extrapolation of Table 6. Finally, with one broker node per host the 3-node cluster is no longer confounded by co-location: REDIS Cluster delivers 1.65 ms median TTI (0.84 ms transport) and KAFKA 14.1 ms (1.05 ms transport) at N=5, both comfortably inside every budget in Table 1.

5.7 Supporting: Consistency–Latency Trade-off (H31, H32)

Stronger durability guarantees cost latency, as hypothesised. For KAFKA, `acks=all` raises median TTI relative to `acks=1`. For REDIS, synchronising the append-only file on every write (`appendfsync always`) raises median TTI by far more than `everysec`, reflecting per-write disk synchronisation. Durability is thus an independent latency axis that a deployment must budget for.

5.8 Supporting: Fairness Controls (RQ3)

The comparison is not biased by payload or protocol: message payloads are near-identical across backends (median ∼170 bytes), and serialization/deserialization overheads are negligible and comparable (∼2.3/2.1 μs for KAFKA, ∼2.4/2.2 μs for REDIS). Mapping latency to the sports-actionability thresholds of Table 1, both backends meet *every* budget in the table— including the tightest, in-play betting at <200 ms—at every concurrency level tested, with roughly an order of magnitude of headroom. The budgets are breached only in the two regimes this paper identifies as the real constraints: the host-saturating event rates of Table 6, and round-trip-bound consumption across a network hop (Section 5.5), where a 20 ms delay puts REDIS four orders of magnitude outside every budget listed until acknowledgements are batched.

6 Discussion

The answer has two halves, and the second is the one we did not expect.

Under load, the answer is deliberately undramatic. Across every event rate a football operator will realistically meet—up to the equivalent of roughly a thousand simultaneous real-time matches—KAFKA and REDIS deliver events in about ten milliseconds, are statistically *equivalent* within one broadcast frame, and leave a goal's win probability stale by ≈0.014 probability-seconds. That is far below any threshold at which a coach, broadcaster or model could act differently. It is a direct empirical rebuttal of the industry framing of a "2-second edge" decided by infrastructure: at football event rates the edge is simply not there, because the data rate is not high enough for the broker to become the binding constraint.

Under *network delay*, that equivalence collapses—and the reason is more interesting than a verdict on either product. The failure is not throughput, not the broker, and not REDIS Streams as a technology. It is a *consumption pattern*: a client that acknowledges each message and blocks for the reply can never drain faster than $1/\mathrm{RTT}$, so once the round trip exceeds the mean event interarrival time the consumer stops keeping up and latency is set by an unbounded, still-growing backlog rather than by the delay. That is why the amplification is a factor of 1,500 rather than a factor of

Table 15. Acknowledgement batching at 20 ms injected delay, $N=1$, true real-time replay, three replications. Read-loop instrumentation is recorded per run.

Arm	TTI median (ms)	Reads in run	Non-empty reads	Messages per read
Per-message acknowledgement	4,138	16	4	106
Batched acknowledgement	103	40	25	16

Replicates: unbatched 4137.9/4133.9/4138.2 ms; batched 103.0/103.3/103.0 ms. Improvement 40.2×.

one: 75 seconds cannot be assembled from 50 ms applied once per event. KAFKA is not immune by virtue of being KAFKA; it is immune because its client already amortises—prefetching records and committing offsets on a timer—so it has no per-record round trip left to expose. We regard the intervention as the load-bearing evidence here: changing one line, so that acknowledgements are batched, recovers a factor of 307 and 882 and restores equivalence, which no observational comparison could have established. That conclusion is bounded by where we could manipulate it: on a multi-host replication the same intervention leaves the collapse untouched (Section 5.6), so the account explains our single-host result rather than network-induced failure in general.

Two things follow. The first is a design rule that transfers beyond this study: with a co-located broker, per-message round trips are free and the pattern is invisible; at a realistic RTT they are the dominant term. Benchmarks conducted on loopback—which is to say most benchmarks—systematically price this at zero, and will therefore certify as equivalent two clients that differ by three orders of magnitude in deployment. The second is a caution about how we ourselves nearly reported the opposite: the threshold $\mathrm{RTT} \approx 1/\lambda$ is a property of the arrival rate, so the 19 ms figure is specific to football's 0.44 events per second per match scaled by our replay. The transferable claim is the relationship, not the number.

The reason is worth stating plainly, because it is easy to lose in a benchmark. Football event data is *sparse*: 0.415 events per second per match on average, though bursting to 2.70 (Section 3.1). A broker that comfortably sustains thousands of events per second is being asked, by a realistic deployment, for a few hundred. Architectural differences that are entirely real—REDIS's single execution thread does serialize concurrent streams, and we measure it doing so at $p = 6.7 \times 10^{-12}$ with complete rank separation—are therefore inert in this domain, because the serialization amounts to 0.34 ms. This is the cleanest illustration we can offer of why statistical significance and practical relevance must be reported together: on the same data, the difference-test rejects equality and the equivalence-test establishes equivalence, and both are correct. We found the effect at all only by decomposing TTI; had we reported the aggregate metric alone, as the streaming-benchmark literature typically does, we would have concluded—and initially did conclude—that no architectural difference existed. We found its true magnitude only by making the concurrent feeds carry *distinct* matches: an earlier design in which every feed replayed the same ten-match merged plan produced an apparent "REDIS degrades with concurrency" result that was really an artefact of packing ten matches into each feed. The lesson generalises beyond our study: a concurrency axis that is secretly a throughput axis will manufacture architectural differences that the domain never experiences.

The practical guidance follows, and it is conditional rather than a ranking. If consumers are co-located with the broker, select on durability semantics, operational familiarity and cost—not on latency, and not on published throughput benchmarks conducted at rates the domain does not reach. If any consumer sits a network hop away, audit the acknowledgement pattern before choosing a product: batch the acknowledgements, and the systems are equivalent again; leave them per-message, and no amount of broker capacity will save the deployment. Provisioning for throughput matters only if aggregate event rate approaches the thousands-of-matches regime, which for event data means a scale no league produces.

The study is equally a methodological cautionary tale. Six independent design defects each reversed the apparent winner before correction (Section 4.6). Cross-process latency benchmarks must share a clock epoch; the load generator must not saturate; the two systems' clients must be configured comparably, since a synchronous KAFKA producer competing against an asynchronous REDIS dispatcher measures the load generator rather than the broker; and a concurrency axis must not covertly vary throughput, or it will manufacture architectural differences the domain never experiences. To these we add a fifth, of a different character. Our first attempt at the batching intervention reported no effect—a clean refutation of our own hypothesis—because the treatment had silently never been applied, and we caught it only by deliberately passing an invalid argument to confirm the manipulation reached the client. A null result from an unapplied treatment is, in the data, indistinguishable from a genuine one; interventions need positive controls. We report all five because the corrected protocol, not merely the corrected numbers, is what makes a streaming comparison honest.

Limitations. First, the testbed bounds the resolution of every reported number. The host's sleep primitive quantises event emission to a ≈15.6 ms grid, which is what produces the several milliseconds of producer scheduling lag in TTI, and the container network path costs ≈1 ms even at rest. We therefore make no claim about sub-millisecond differences in TTI, and confine architectural comparison to transport, where both backends are stamped identically. A Linux host with a high-resolution timer and host networking would lower both floors; we would expect it to sharpen the transport separation we report in Table 5 rather than remove it, but we have not verified that, and it is the most obvious replication target.

Second, all experiments run on a *single commodity host*. This cleanly isolates the single-node comparison but *confounds* the 3-node cluster experiments: co-locating

six broker containers with the load generator saturates the host, and cluster-specific client behaviour (KAFKA topic-creation/metadata blocking; REDIS cluster redirection) inflates latency for reasons unrelated to clustering. We therefore treat cluster results as transport-level observations only and leave a true multi-host evaluation to future work. This bears directly on our network results: the delay in Section 5.5 is *synthetic*, injected with `tc netem` on a single host, so it reproduces the latency of a network hop but not its jitter, loss, reordering or bandwidth limits. The intervention (P4) establishes the causal direction firmly—the same delay, with only the acknowledgement pattern changed, yields a 307–882× difference—but it does not survive replication on genuinely separate hosts, where the collapse recurs and batching does not repair it (Section 5.6). We therefore treat the synthetic-delay result as establishing the mechanism in a controlled setting, not as a general account of network-induced failure. Relatedly, the threshold $\mathrm{RTT}_{\mathrm{crit}} \approx 1/\lambda$ is a function of arrival rate, so the specific figure of ≈19 ms belongs to our replay conditions ($\lambda \approx$ 53 events/s per feed) and not to football in general; it is the *relationship* that transfers, and a slower or faster feed moves the knee accordingly. Third, the win-probability model is a transparent, calibrated *proxy* for the unreleased state-of-the-art model Robberechts et al. (2021); our decision-staleness magnitudes are indicative rather than exact, although the null result is robust to the proxy's precise calibration because both backends are evaluated against the identical model. Fourth, feeds replay recorded matches rather than consuming genuinely live sources. We mitigate the two damaging forms of this directly: each feed carries a *distinct* match, which is what revealed the throughput confound of Section 5.3, and the headline condition is repeated at true 1× speed, which reproduces the compressed result. What remains unmodelled is the upstream annotation pipeline, and it deserves more than a passing caveat because it is the *dominant term*. Live football event data is produced by human or vendor annotation, and published accounts of such pipelines put the lag from on-pitch action to available event in the range of seconds Pappas et al. (2020); Sports (2023). Set against that, the entire range this paper measures—from 0.4 ms of broker transport to the 40 ms equivalence margin—is two to four orders of magnitude smaller.

A reader is entitled to draw the sharp conclusion: if annotation contributes seconds, then for a co-located deployment the answer to "does infrastructure choice degrade an in-play decision?" is *no*, and it is knowable *a priori* from the ratio of the two terms. We accept that, and we think it is the honest reading of our own null. What the measurements add is, first, the quantification that licenses the ratio argument at all—nobody had measured the transport term against a decision-relevant scale—and second, the demonstration that the conclusion is *conditional*: once a network hop and a round-trip-bound consumer are present, transport stops being negligible and reaches 75 seconds, which is far larger than any annotation lag and would dominate rather than be dominated. The contribution is therefore better stated as locating the boundary at which the infrastructure term stops being ignorable than as discovering that it usually is. Our figures bound the *transport* contribution to staleness, not end-to-end feed freshness, and an end-to-end account would need to measure annotation directly—which the open data does not permit, since StatsBomb events carry no annotation timestamps. Fifth, our equivalence claim is bounded by the margin we chose: TOST establishes that the backends differ by less than one broadcast frame, not that they are identical, and a stricter margin (say, one millisecond) would not be supported by these data. We regard a frame as the operationally meaningful threshold, but readers applying the result under tighter budgets should re-test at their own margin. Finally, the study covers football event data specifically; the conclusion that broker choice is immaterial follows from that domain's low event rate and should not be transferred to high-frequency telemetry such as ball- or limb-tracking streams, where the throughput regime is entirely different.

7 Conclusion

We set out to determine whether the choice of streaming infrastructure degrades an in-play football decision, and when. After correcting six design defects that had each reversed the apparent winner, the answer proves to be conditional — and the condition is neither the backend nor the load, but the deployment topology and the consumption pattern it exposes.

With components co-located, KAFKA and REDIS are statistically *equivalent* at every concurrency level tested (≈10 ms median TTI, ≈0.014 probability-seconds of win-probability staleness; TOST against a one-broadcast-frame margin), and neither shows a concurrency effect. Single-node REDIS does serialize concurrent streams, but only above an aggregate event rate far above the 12 simultaneous kick-offs the corpus actually contains—orders of magnitude beyond a busy league weekend, because football event data arrives at only 0.44 events per second per match.

Introduce a network hop and that equivalence collapses by three orders of magnitude, and the mechanism turns out to be an implementation choice rather than a property of either system. A consumer that acknowledges each message and waits for the reply is capped at $1/\mathrm{RTT}$; when that ceiling drops below the arrival rate—at $\mathrm{RTT} \approx 1/\lambda \approx 19$ ms under our conditions, a threshold a fine delay sweep confirms—it accumulates a backlog that grows for as long as the match lasts. Batching those acknowledgements, a single line, recovers factors of 307 and 882 and restores equivalence with KAFKA—on that testbed. On a multi-host replication with a genuine inter-VM network the collapse recurs but the repair does not, and the accumulating-backlog signature the account depends on is absent (Section 5.6). The mechanism is therefore demonstrated where we could manipulate it and unresolved where we could not, which is a weaker claim than we set out to make and the one the data supports.

By translating measured delivery latency into win-probability staleness through the Age-of-Information lens, we provide what is, to our knowledge, the first measured account of *whether and when* streaming-infrastructure latency is decision-relevant for live football, on open data with open code. The practical message is conditional and, we think, more useful than a verdict: if consumers are co-located with the broker, choose on durability and operational grounds, because latency will not distinguish the

systems; if any consumer sits a network hop away, audit the consumption pattern before the product, because a client that waits for a reply per message converts ordinary network latency into seconds of decision staleness, and batching those replies removes the problem. The methodological message is twofold: cross-process streaming benchmarks demand a shared clock, a non-saturating load generator and symmetrically configured clients; a concurrency axis that is secretly a throughput axis will manufacture architectural differences the target domain never encounters; a co-located testbed will conceal the one difference that matters most, because the cost of a round-trip-bound design is zero until the round trip is real; and an intervention whose treatment silently fails to apply produces a null result indistinguishable from a genuine refutation—our first attempt at the batching experiment did exactly that, and was caught only by deliberately passing an invalid argument to confirm the manipulation had reached the client.

Data and Code Availability

All code, replay plans, aggregated results and the analysis pipeline are openly available at https://github.com/Gustolandia/streaming-latency-sports, archived at [DOI to be inserted on release]. The event data are the StatsBomb open dataset (1. Bundesliga 2023/24), used under its CC BY-NC 4.0 licence; the raw event JSONs are not redistributed here but are re-fetchable from a pinned upstream commit with the script scripts/fetch_statsbomb_events.sh, so the corpus is reconstructible exactly. Every run records its git commit and per-file SHA-256 in a meta.json, and reproducibility/MANIFEST.json pins the checksums of all code and configuration used to produce the reported results.

Acknowledgements

Using StatsBomb open dataset. Open-source Apache Kafka and Redis projects. MIT License.

References

Abadi D et al. (2012) The PACELC theorem: Consistency in distributed systems. Extension of CAP theorem addressing latency-consistency trade-offs.

Anonymous (2025) Analysis of design patterns and benchmark practices in Apache Kafka event-streaming systems. *arXiv preprint* URL https://arxiv.org/html/2512.16146v1. Accessed: June 15, 2026.

Arasu A, Cherniack M, Galvez E, Maier D, Maskey AS, Ryvkina E, Stonebraker M and Tibbetts R (2004) Linear road: A stream data management benchmark. In: *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*. pp. 480–491.

Brewer EA (2012) Cap twelve years later: How the "rules" have changed. *IEEE Computer* 45(2): 23–29.

Carbone M et al. (2015) A performance comparison of big data stream processing frameworks. *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data* .

Cohen J (1988) *Statistical Power Analysis for the Behavioral Sciences*. 2nd edition. Lawrence Erlbaum Associates.

Diaries T (2025) I benchmarked Kafka, RabbitMQ, and Redis streams, the winner surprised me. URL https://medium.com/@ThreadSafeDiaries/i-benchmarked-kafka-rabbitmq-and-redis-streams-the-winner-surprised-me-cf3f484eb7b2. Accessed: June 15, 2026.

Gai K and Qiu M (2020) Kafka on kubernetes: performance evaluation and optimization. *Cluster Computing* .

He W et al. (2020) Performance evaluation of apache kafka for big data streaming. *IEEE Access* 8: 123456–123467.

Hesse G, Matthies C, Perscheid M, Uflacker M and Plattner H (2021) ESPBench: The enterprise stream processing benchmark. In: *Proceedings of the ACM/SPEC International Conference on Performance Engineering*. pp. 201–212.

Holm S (1979) A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6(2): 65–70.

JusDB (2025) Redis streams vs Kafka: Event streaming architecture comparison 2025. URL https://www.jusdb.com/blog/redis-streams-vs-kafka-event-streaming-comparison-2025. Accessed: June 15, 2026.

Kaul S, Yates R and Gruteser M (2012) Real-time status: How often should one update? In: *2012 Proceedings IEEE INFOCOM*. IEEE, pp. 2731–2735. DOI:10.1109/INFCOM.2012.6195689.

Kreps J, Narkhede N and Rao J (2011) Kafka: a distributed messaging system for log processing. *Proceedings of the 6th ACM Symposium on Networked Systems Design and Implementation* .

Link D et al. (2021) Deep learning for player tracking and event detection in sports. *IEEE Access* 9: 98765–98780.

Liu H, Hopkins W, Gomez AM and Molinuevo JS (2013) Inter-operator reliability of live football match statistics from opta sportsdata. *International Journal of Performance Analysis in Sport* 13(3): 803–821.

OpenMessaging Project, Linux Foundation (2018) The OpenMessaging benchmark framework. Open standard, multi-platform messaging performance benchmark. Available at openmessaging.cloud/docs/benchmarks/.

OptiView D (2025) What is low latency video streaming?: The complete guide. URL https://optiview.dolby.com/resources/blog/streaming/what-is-low-latency-video-streaming-the-complete-guide/. Accessed: June 15, 2026.

Pandey R et al. (2021) Comparative analysis of apache kafka and redis streams for real-time data processing. *Journal of Big Data* 8(1): 1–20.

Pappas I et al. (2020) Real-time football analytics: requirements and architecture. *Journal of Sports Analytics* 6(2): 89–104.

Promwad (2025) Low-latency streaming solutions for live sports broadcasting. URL https://promwad.com/news/low-latency-streaming-solutions-live-sports-broadcasting. Accessed: June 15, 2026.

Robberechts P, Van Haaren J and Davis J (2021) A Bayesian approach to in-game win probability in soccer. In: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. pp. 3512–3521. DOI:10.1145/3447548.3467194.

Sanfilipe S (2017) Redis streams: a new data type for real-time streaming. *Redis Labs Blog* URL https://redis.io/topics/streams-intro.

Shukla A, Chaturvedi S and Simmhan Y (2017) RIoTBench: An IoT benchmark for distributed stream processing systems. *Concurrency and Computation: Practice and Experience* 29(21): e4257.

Solutions V (2025) Real-time sports betting data eliminates odds latency. URL https://www.v2solutions.com/blogs/real-time-sports-betting-data-odds-latency/. Accessed: June 15, 2026.

Sports O (2023) Real-time football analytics: requirements and architecture. URL https://www.optasports.com/. Industry latency requirements; Accessed: June 15, 2026.

StatsBomb (2023) Statsbomb open data. *GitHub Repository* URL https://github.com/statsbomb/open-data.

Stonebraker M, Cetintemel U and Zdonik S (2005) The 8 requirements of real-time stream processing. *ACM SIGMOD Record* 34(4): 42–47.

Tucker P, Tufte K, Papadimos V and Maier D (2008) NEXMark: A benchmark for queries over data streams. Technical report, OGI School of Science and Engineering, Oregon Health and Science University.

Ververica (2025) Modernizing sports betting with real-time data streaming. URL https://www.ververica.com/blog/modernizing-sports-betting-technology-to-empower-live-odds. Accessed: June 15, 2026.

Wright M et al. (2022) Machine learning for real-time sports analytics: a survey. *ACM Computing Surveys* 55(8): 1–35.

Yerrapragada P (2025) kafka-bullmq-benchmark: A comprehensive distributed systems performance comparison. URL https://github.com/praneethys/kafka-bullmq-benchmark. Includes white paper with methodology; Accessed: June 15, 2026.

Zhang L and Lee VC (2022) Redis streams vs apache kafka: a performance comparison for iot applications. *Sensors* 22(15): 5678.

Zhang M et al. (2021) Time-to-insight: a metric for evaluating streaming analytics systems. *IEEE Transactions on Big Data* .